

Tree Algorithms

Sean

August 26, 2026



Outline

- Tree model and facts
- Rooted trees
- DFS on trees
- Subtree sizes
- Euler tour order
- Tree diameter
- Tree problem patterns

Why trees?

- Trees are connected graphs with no cycles
- They model hierarchy, recursion, dependencies, and choices with one unique path
- Their structure removes many graph complications: no cycles, no alternate routes
- Most tree algorithms are DFS plus the right information flowing in the right direction
- This lecture builds on graph traversal from Graph Basics



1 Tree model and facts

2 Rooted trees

3 DFS on trees

4 Subtree sizes

5 Euler tour order

6 Tree diameter

7 Tree problem patterns



Tree model and facts

What is a tree?

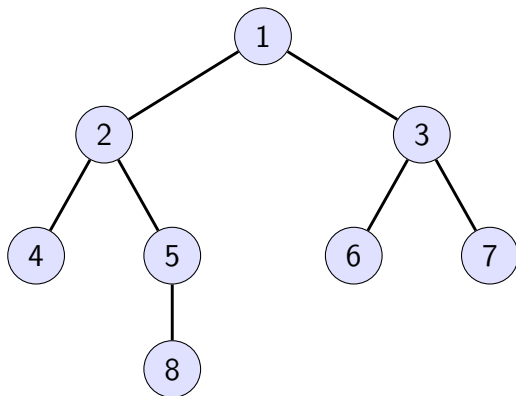
- A tree is a connected graph with no cycles
- If a tree has n vertices, it has exactly $n - 1$ edges
- Between any two vertices, there is exactly one simple path
- Removing one edge splits the tree into two components



Three useful tree tests

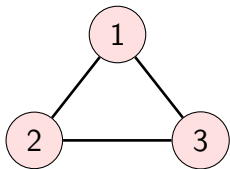
- Connected + no cycles $\implies$ tree
- Connected + exactly $n - 1$ edges $\implies$ tree
- No cycles + exactly $n - 1$ edges $\implies$ tree
- For an undirected graph, any two of these facts force the third

Example tree

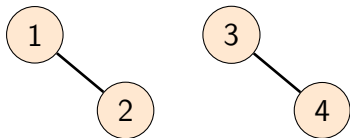


- $n = 8$ vertices and $n - 1 = 7$ edges
- Every pair of vertices has exactly one path between them

What breaks the tree property?



A cycle gives multiple paths



Disconnected pieces give no path

1 Tree model and facts

2 Rooted trees

3 DFS on trees

4 Subtree sizes

5 Euler tour order

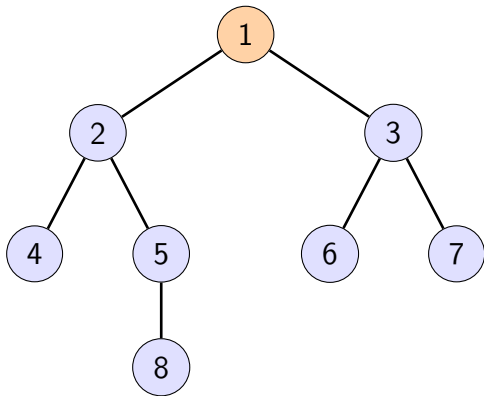
6 Tree diameter

7 Tree problem patterns



Rooted trees

Rooting a tree



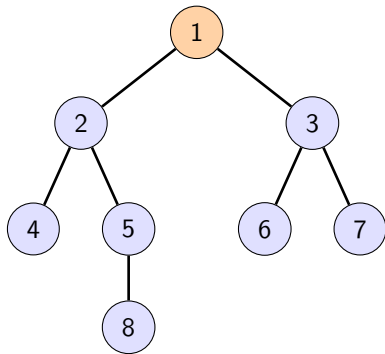
- Choose one vertex as the root
- DFS gives every other vertex a parent
- Distance from the root becomes depth
- Subtrees become meaningful only after rooting

Rooted tree vocabulary

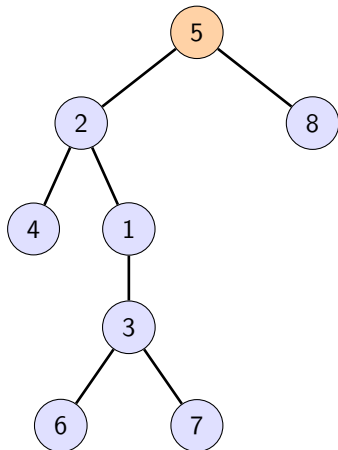
Term	Meaning
Root	Vertex with no parent
Parent	Previous vertex on the path to the root
Child	Neighbor whose parent is the current vertex
Depth	Number of edges from the root
Subtree	A vertex and all descendants below it



Same tree, different root



Root at 1: subtree of 2
has 4 vertices



Root at 5: subtree of 2
now has 6 vertices

1 Tree model and facts

2 Rooted trees

3 DFS on trees

4 Subtree sizes

5 Euler tour order

6 Tree diameter

7 Tree problem patterns



DFS on trees

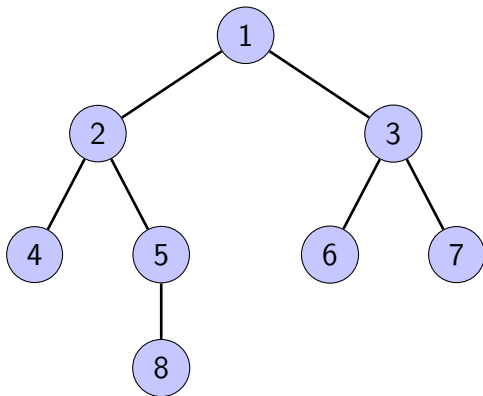
Tree DFS pattern

```
1 void dfs(int u, int p) {  
2     for (int v : adj[u]) {  
3         if (v == p) continue;  
4         dfs(v, u);  
5     }  
6 }
```

- The parent check replaces `visited` in a tree
- There are no cycles, so the only immediate back edge is to the parent



DFS trace



- Start at root 1
- Follow the first unvisited child recursively
- Return when a child subtree is finished
- DFS preorder:
1, 2, 4, 5, 8,
3, 6, 7

Parent and depth

```
1  vector<int> parent(n + 1), depth(n + 1);
2
3  void dfs(int u, int p) {
4      parent[u] = p;
5      for (int v : adj[u]) {
6          if (v == p) continue;
7          depth[v] = depth[u] + 1;
8          dfs(v, u);
9      }
10 }
```



Preorder and postorder

Preorder	Postorder
Process a vertex when DFS first enters it.	Process a vertex after all child subtrees return.
■ Useful for parent, depth, entry time ■ Example: 1, 2, 4, 5, 8, 3, 6, 7	■ Useful for subtree DP ■ Example: 4, 8, 5, 2, 6, 7, 3, 1

1 Tree model and facts

2 Rooted trees

3 DFS on trees

4 Subtree sizes

5 Euler tour order

6 Tree diameter

7 Tree problem patterns



Subtree sizes

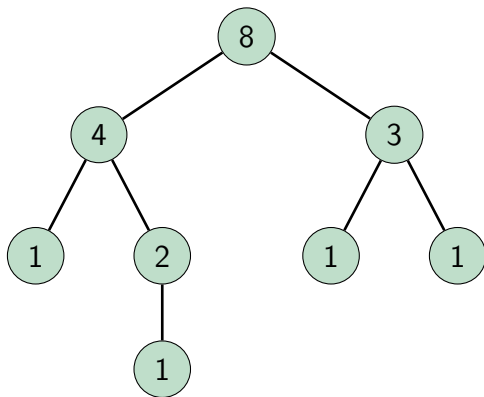
Subtree sizes

```
1  vector<int> sub(n + 1);
2
3  void dfs(int u, int p) {
4      sub[u] = 1;
5      for (int v : adj[u]) {
6          if (v == p) continue;
7          dfs(v, u);
8          sub[u] += sub[v];
9      }
10 }
```

- Compute children first, then add their answers into the parent



Subtree-size trace



- Leaves return size 1
- Internal vertices add child answers
- Postorder matters: children first, parent after
- Final subtree sizes are shown inside the nodes

1 Tree model and facts

2 Rooted trees

3 DFS on trees

4 Subtree sizes

5 Euler tour order

6 Tree diameter

7 Tree problem patterns



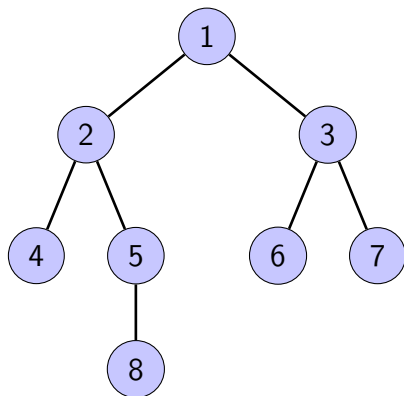
Euler tour order

Flattening a tree

- DFS can assign each vertex an entry time $\text{tin}[u]$: “time in”
- If we list vertices when DFS first enters them, every subtree becomes contiguous
- $\text{tout}[u]$ marks the last flattened index that still belongs to the subtree
- Then many subtree questions become ordinary array range questions
- This is one of the most useful bridges from trees to data structures



Euler tour trace



1	2	4	5	8	3	6	7
0	1	2	3	4	5	6	7

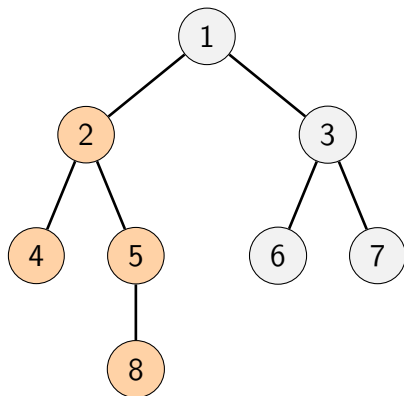
- DFS entry order:
1, 2, 4, 5, 8, 3, 6, 7
- The array index is $\text{tin}[u]$: “time in”
- Children stay next to their parent subtree

Euler tour code

```
1  vector<int> tin(n + 1), tout(n + 1), order;
2  int timer = 0;
3
4  void dfs(int u, int p) {
5      tin[u] = timer++; // time in
6      order.push_back(u);
7
8      for (int v : adj[u]) {
9          if (v == p) continue;
10         dfs(v, u);
11     }
12
13     tout[u] = timer - 1; // last index in u's subtree
14 }
```



A subtree becomes an interval



1	2	4	5	8	3	6	7
0	1	2	3	4	5	6	7

- Subtree of 2: vertices 2, 4, 5, 8
- Interval: $\text{tin}[2] \dots \text{tout}[2] = 1..4$
- Now subtree sums or updates can use array tools

1 Tree model and facts

2 Rooted trees

3 DFS on trees

4 Subtree sizes

5 Euler tour order

6 Tree diameter

7 Tree problem patterns



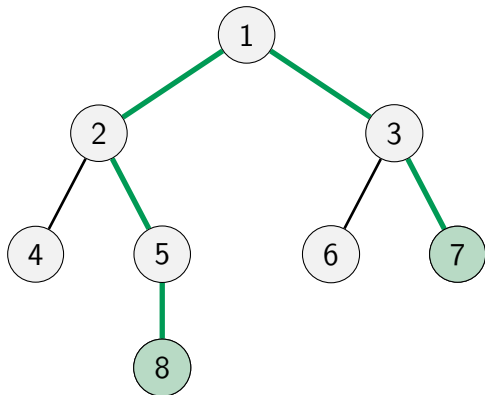
Tree diameter

Tree diameter

- The diameter is the longest path between two vertices
- Run BFS or DFS from any vertex to find a farthest vertex a
- Run BFS or DFS from a to find a farthest vertex b
- The distance from a to b is the tree diameter



Two-sweep diameter trace



- From any start, one farthest vertex is a diameter endpoint
- Starting from that endpoint finds the opposite endpoint
- Here the diameter is the path from 8 to 7
- Length: 5 edges

1 Tree model and facts

2 Rooted trees

3 DFS on trees

4 Subtree sizes

5 Euler tour order

6 Tree diameter

7 Tree problem patterns



Tree problem patterns

Three common tree moves

Flow	Typical values	Use it for
Top-down	parent, depth	distances from root, ancestors later
Bottom-up	subtree size, DP state	aggregate answers from children
Flatten	tin, tout	subtree queries with array tools



Common tree mistakes

- Forgetting to skip the parent, causing infinite recursion
- Updating a parent before child answers are ready
- Treating a subtree as meaningful before choosing a root
- Mixing up vertex count and edge count on path lengths
- Assuming every tree problem needs a new algorithm instead of reusing one of the three flows



Example: Subordinates

CSES: Subordinates

For every employee, count how many employees are below them in the company hierarchy.

- The company chart is already a rooted tree
- If $\text{sub}[u]$ counts the whole subtree, then the answer is $\text{sub}[u] - 1$
- Leaves return 1; managers add the answers from their direct reports
- One postorder DFS solves every employee in linear time



What belongs in the next tree lecture?

- Lowest common ancestor and binary lifting
- Distances between arbitrary vertices
- Rerooting dynamic programming
- More advanced tree DP patterns



Practice problems

- CSES: Subordinates – subtree sizes
- CSES: Subtree Queries – Euler tour + range queries
- CSES: Tree Diameter – longest path in a tree
- CSES: Tree Distances I – distances from diameter endpoints
- CSES: Tree Matching – tree DP preview



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

